



Auditoría Exhaustiva de Código, Rendimiento y UX

Diagnóstico exhaustivo sobre el estado del frontend de la plataforma **Atlas Gym**: análisis de tamaño de bundle y empaquetado, arquitectura del estado global, bugs silenciosos en flujos de cuotas, brechas funcionales para gimnasios reales y hoja de ruta priorizada de optimizaciones.

TESTS UNITARIOS

92 / 92

100% pasando (Vitest)

CHUNK JS INICIAL

1.22 MB

Por catálogo estático

EJERCICIOS CON GIFS

1.500+

Mapeados en español

TYPESCRIPT

0 errores

Strict check limpio

1. ¿Qué está mal o presenta riesgo técnico? (Bugs silenciosos)

1.1. Bundle Inicial de 1.2 MB por catálogo de 1.500 ejercicios estático Crítico / Performance

Causa: `lib/data/exercises.json` (1.17 MB) se importa estáticamente en `lib/catalogo-ejercicios.ts` y se vuelca en `lib/store.tsx`.

Efecto: El chunk generado por Turbopack pesa **1.217.607 bytes (1.22 MB)** (`1e831b1vurg9.js`), impactando negativamente en la velocidad de carga inicial. Además, el `useEffect` escribe todo ese volumen en `localStorage`, ocupando el 30% del límite total del navegador del cliente.

Contradicción: Ya existe el endpoint `app/api/ejercicios/route.ts` listo para filtrar y paginar, pero **ningún componente del frontend lo está consultando**.

1.2. Re-renders masivos por Context Monolítico (lib/store.tsx) Arquitectura

Causa: `useAppData()` contiene en un solo proveedor: alumnos, pagos, rutinas, videos, planes, avisos, sesiones y autenticación.

Efecto: Cuando un alumno marca una serie como hecha en su rutina (`guardarSesion`) o se modifica un aviso, **todos los componentes que consumen el hook sufren re-render** (Sidebar, AppShell, Notificaciones, etc.). Conviene desacoplar en contextos especializados (ej: `AuthContext` vs `GymDataContext`).

1.3. Desajuste en el filtrado de cuotas mensuales (app/finanzas/page.tsx)

Lógica de Negocio

Causa: La línea 324 filtra los pagos del mes con `pagos.filter((p) => p.fecha.slice(0, 7) === mes)`. Esto filtra por la fecha de cobro (`fecha`) y no por el mes correspondiente a la cuota (`periodoMes`).

Impacto: Si un alumno paga la cuota de Agosto el 2 de Septiembre, ese pago no aparece en la planilla de Agosto y se imputa a Septiembre. Además, el formulario para registrar pagos no permite elegir qué mes se abona; autoasigna la fecha del día.

1.4. Búsqueda de Alumnos limitada exclusivamente a Nombre

UX Recepción

En `app/alumnos/page.tsx`, el buscador solo evalúa `a.nombre.toLowerCase()`. En la recepción de un gimnasio es habitual buscar por **DNI**, **email** o **celular**. Si se escribe el DNI, la tabla arroja 0 resultados.

1.5. Disparidad en patrones de Modales y Accesibilidad (a11y)

Accesibilidad

Existen 3 implementaciones distintas: `<dialog>` con `showModal()` (Alumno, Perfil, Aviso), `<dialog open>` sin API modal (Planes), y `<div fixed>` (Finanzas, Rutinas, Videoteca). Estos últimos no atrapan el foco del teclado ni cierran de forma estándar con la tecla `Esc`.

2. ¿Qué funcionalidades faltan para la operación real?

FUNCIONALIDAD	MOTIVO / NECESIDAD OPERATIVA	PRIORIDAD	ESFUERZO
Control de Asistencia en 1 Click	El modelo tiene <code>ultimaAsistencia</code> , pero no hay botón para marcar "Presente hoy" ni en la tabla de alumnos ni en su ficha.	Alta	Bajo
Exportación a Excel / CSV	El administrador necesita descargar la planilla de pagos del mes y la lista de deudores para liquidaciones y rendición contable.	Alta	Bajo
Generación Mensual de Cuotas	Al iniciar cada mes, el sistema debe emitir la cuota PENDIENTE a todos los socios activos para controlar moras el día 10 automáticamente.	Media	Medio
Toasts / Alertas de Feedback	Al dar de alta un alumno o registrar un cobro, los modales cierran sin confirmación visual explícita ("Pago registrado con éxito").	Media	Bajo
PWA Manifest Real	Existe un tutorial de instalación en el login, pero falta configurar <code>manifest.json</code> y los meta-tags para instalación nativa.	Media	Bajo
Filtro por Medio de Pago	Permitir al dueño filtrar ingresos por Efectivo, Transferencia o Mercado Pago para hacer el cierre de caja diario.	Deseable	Bajo

3. ¿Qué código se debe limpiar para máxima velocidad y mantenibilidad?

Modularización de Archivos Gigantes

Archivos monolíticos que concentran demasiada lógica y deberían dividirse en submódulos:

- `app/rutinas/page.tsx`: 1.870 líneas
- `app/finanzas/page.tsx`: 1.079 líneas
- `app/videoteca/page.tsx`: 635 líneas
- `app/alumnos/page.tsx`: 564 líneas

Componentes Duplicados

Elementos visuales idénticos declarados dos veces en vez de centralizarse:

- **MetricCard**: Código duplicado en Alumnos y Finanzas. Centralizar en `components/ui/metric-card.tsx`.
- **Badges de Estado**: Estilos repetidos en 3 archivos.
- **next/image**: En `sidebar.tsx` reemplazar `<img>` para evitar layout shifts.

4. Hoja de Ruta Sugerida (Roadmap)

1 Fase 1 — Optimización del Bundle y Activación de la API de Ejercicios

Conectar el frontend al endpoint `/api/ejercicios` para buscar y paginar bajo demanda. Desacoplar los 1.500 ejercicios del estado sincrónico de arranque. El bundle se reduce de 1.2 MB a ~200 KB y se liberan 1.5 MB de `localStorage`.

2 Fase 2 — Correcciones Operativas (Alumnos & Finanzas)

Extender el buscador de Alumnos a DNI, email y celular. Incorporar selector de mes en cobros (`periodoMes`) y unificar el cierre con tecla `Esc` en todos los modales.

3 Fase 3 — Funcionalidades de Alto Valor para el Negocio

Implementar el botón de asistencia rápida ("Presente hoy" en 1 click), exportación a planilla Excel/CSV para el contador y feedback con notificaciones toast.

4 Fase 4 — Limpieza Arquitectónica y Modularización

Dividir `app/rutinas/page.tsx` (1.870 líneas) en componentes modulares limpios y centralizar componentes UI repetidos.